

شرح طبقة Application

مشروع: **Modiaf.AI.Arab.Hotel** · إطار: .NET 10 + ABP 10.3

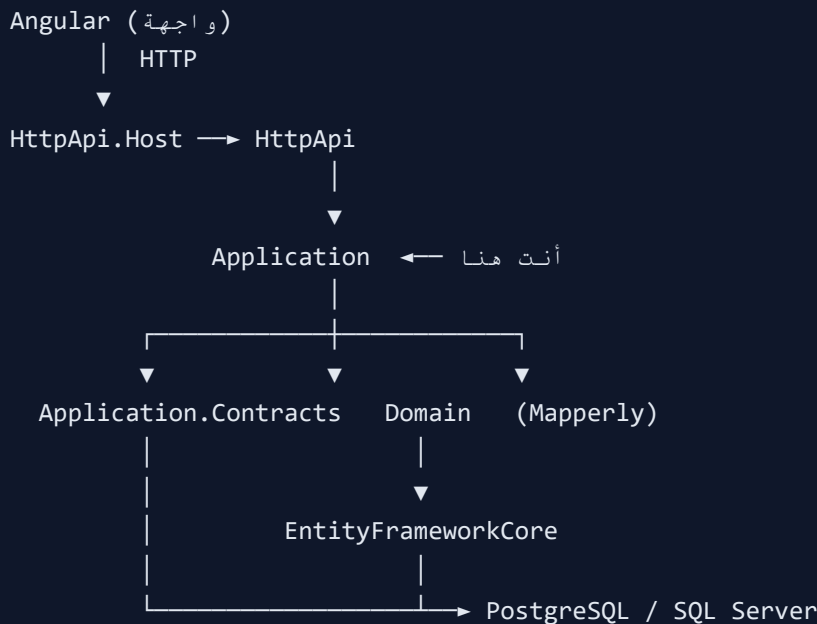
المسار: `aspnet-core/src/Modiaf.AI.Arab.Hotel.Application`

1. ما هي طبقة Application؟

طبقة **Application** هي طبقة **منطق التطبيق (Use Cases)**. تجمع بين واجهات المستخدم (Angular) وطبقة Domain (الكيانات وقاعدة البيانات). كل عملية يطلبها المستخدم — تسجيل دخول، CRUD غرفة، حفظ ترجمة — تُنفَّذ هنا عبر **App Service**.

قاعدة مهمة: Application لا تتحدث مع قاعدة البيانات مباشرةً دائماً، بل عبر `IRepository` من Domain أو عبر `Store` مخصّص. ولا تعرّف `Controllers` يدوياً — ABP ينشئ REST API تلقائياً.

2. مكانها في المعمارية



3. التبعية (Dependencies)

يعتمد على	الغرض
Application.Contracts	واجهات I*AppService و DTOs
Domain	الكيانات، Domain Services، IRepository
ABP Modules	...Identity, Account, Permissions, Settings

لا يعتمد على: EntityFrameworkCore، HttpApi، Angular.

4. هيكل المجلدات

```
Modiaf.Al.Arab.Hotel.Application/
├── HotelApplicationModule.cs      ← نقطة دخول الطبقة
├── HotelAppService.cs            ← أساسي لكل الخدمات Class
├── HotelApplicationMappers.cs    ← Mapperly (Entity ↔ DTO)
├──
├── Entities/                    ← كيانات الفندق CRUD: نمط قديم
│   ├── Rooms.cs                → RoomAppService
│   ├── Bookings.cs             → BookingAppService
│   ├── Floors.cs
│   ├── GuestRegistries.cs
│   ├── IdentityTypes.cs
│   └── RoomTypes.cs
├──
├── GeneralCodes/                ← Feature folder: نمط جديد
│   ├── GeneralCodesAppService.cs ← واجهة رفيعة
│   └── GeneralCodesStore.cs      ← Mapping + منطق
├──
├── HotelAuth/                   → LoginAsync
├── HotelUsers/                  → مستخدمي الفندق CRUD
├── HotelSettings/               → إعدادات النظام
├── PaymentMethods/
├── CreditCardTypes/
├── HotelChains/
├── UiTranslations/
│   ├── UiTranslationsBlobAppService.cs
│   └── UiTranslationsFileStore.cs
└── DatabaseAdmin/               → Backup + Migrate
```

5. أنماط التصميم المستخدمة

5.1 App Service + CrudAppService (الكيانات القديمة)

مثل `RoomAppService` — يرث من `CrudAppService` ويوفر `Get/List/Create/Update/Delete` جاهزة.

```
[AllowAnonymous]
public class RoomAppService(IRepository<Room, int> repository)
    : CrudAppService<Room, RoomDto, int, ...>(repository),
    IRoomAppService
{
    protected override RoomDto MapToGetOutputDto(Room entity) { ... }
}
```

5.2 App Service + Store (الميزات الجديدة)

مثل App Service — `GeneralCodes` رفيع، والمنطق في `GeneralCodesStore`.

الطبقة	الملف	الدور
Contracts	<code>IGeneralCodesAppService</code>	عقد API للواجهة
Contracts	<code>IGeneralCodesStore</code>	عقد التخزين
Application	<code>GeneralCodesAppService</code>	يمرر الطلبات للـ Store
Application	<code>GeneralCodesStore</code>	Repository + Validation + DTO Mapping

5.3 File Store (الترجمات)

`UiTranslationsFileStore` يقرأ/يكتب ملفات JSON على القرص — لا قاعدة بيانات.

5.4 Mapperly

ملف `HotelApplicationMappers.cs` يحول `Entity ↔ DTO` تلقائياً لبعض الكيانات (`Room`, `Booking`, `...Floor`).

6. جدول App Services في المشروع

الخدمة	المجلد	العمليات الرئيسية	API (تقريبي)
RoomAppService	Entities/Rooms.cs	CRUD + ResetAllStatusesAsync	api/app/room/
BookingAppService	Entities/Bookings.cs	CRUD حجوزات	api/app/booking/
FloorAppService	Entities/Floors.cs	CRUD طوابق	api/app/floor/
GuestRegistryAppService	Entities/GuestRegistries.cs	CRUD + SaveProfileAsync	api/app/guest-/registry
HotelAuthAppService	/HotelAuth	LoginAsync	api/app/hotel-/auth/login
HotelAppUserAppService	/HotelUsers	CRUD مستخدمين	api/app/hotel-/app-user
HotelSettingsAppService	/HotelSettings	Get/Update إعدادات	api/app/hotel-/settings
GeneralCodesAppService	/GeneralCodes	CRUD أكواد عامة حسب Category	api/app/general-/codes
PaymentMethodAppService	/PaymentMethods	CRUD	api/app/payment-/method
CreditCardTypeAppService	/CreditCardTypes	CRUD	api/app/credit-/card-type
HotelChainAppService	/HotelChains	CRUD	api/app/hotel-/chain
UiTranslationsBlobAppService	/UiTranslations	Get/Update JSON ترجمات	api/app/ui-/translations-blob
HotelDatabaseAdminAppService	/DatabaseAdmin	Backup + UpdateDatabase	api/app/hotel-/database-admin

7. تدفق طلب typical (مثال: GeneralCodes)

```
1. Angular → GET /api/app/general-codes?category=nationalities
2. ABP → GeneralCodesAppService.GetListAsync()
3. App Svc → store.GetByCategoryAsync("nationalities")
4. Store → repository.GetListAsync(x => x.Category == ...)
5. EF Core → SELECT * FROM AppGeneralCodeItems WHERE Category = ...
6. Store → Map Entity → GeneralCodeItemDto
7. JSON → [{ id, name, category, ... }] → Angular
```

8. Class HotelAppService الأساسي

```
public abstract class HotelAppService : ApplicationService
{
    protected HotelAppService()
    {
        LocalizationResource = typeof(HotelResource);
    }
}
```

يوفر: `CurrentUser` , `GuidGenerator` , `ObjectMapper` , التوطين، وغيرها من خدمات ABP.

9. Application.Contracts vs Application

Application	Application.Contracts
تطبيق الواجهات <code>AppService*</code>	واجهات <code>I*AppService</code>
Repository + استدعاء + Mapping + Validation	DTOs (Input/Output)
<code>[Authorize]</code> أو <code>[AllowAnonymous]</code>	Permissions
لا يُستورد من Angular مباشرة	يُشار إليه من Angular + <code>HttpApi.Client</code>

10. كيف يصل Angular لـ Application؟

1. ABP في `HttpApi.Host` يفعّل `Conventional Controllers`.

2. كل `IApplicationService` في `Application Assembly` يصبح REST API.
3. Angular يستدعي `.../api/app/` (أو عبر Cloudflare Worker proxy).
4. لا حاجة لكتابة Controller يدوي لكل Feature.

11. أمثلة مهمة بالتفصيل

HotelAuthService — تسجيل الدخول

- يقرأ `HotelAppUser` من `Repository`.
- يقارن `UserName/Password` (نصّي حالياً).
- يرجع `HotelLoginResultDto` مع `Role` و `LandingPagePath`.
- `[AllowAnonymous]` — متاح بدون `Token`.

UiTranslationsBlobAppService — ترجمات الواجهة

- `GetAsync` — يجمع 6 ملفات (`ar, en, fr, id, tr, am`) JSON في `payload` واحد.
- `UpdateAsync` — يكتب على `HttpApi.Host/UiTranslations/*.json`.
- المصدر الافتراضي: `Domain.Shared/Localization/*.json` (مضمّن).

GeneralCodesStore — الأكواد العامة

- جدول واحد `AppGeneralCodeItems` مفتاحه `Category`.
- 17 فئة: `...nationalities, room-views, hotel-currency`.
- `Validation: Category` يجب أن يكون من `GeneralCodesCategories.All`.

HotelApplicationModule .12

```
[DependsOn(
    typeof(HotelDomainModule),
    typeof(HotelApplicationContractsModule),
    typeof(AbpIdentityApplicationModule),
    ...)]
public class HotelApplicationModule : AbpModule
{
    public override void ConfigureServices(...)
    {
        context.Services.AddMapperlyObjectMapper<HotelApplicationModule>();
    }
}
```

13. ملخص سريع

السؤال	الجواب
أين منطق الأعمال؟	Application (App Services + Stores)
أين DTOs؟	Application.Contracts
أين Entities؟	Domain
أين SQL؟	EntityFrameworkCore
أين REST API؟	يُنشأ تلقائياً من Application
كيف أضيف Feature؟	Contracts (DTO+Interface) → Application (Service) → Domain (Entity) → EF